

1.1 Getting Started with R and RStudio

Install R and RStudio, run a script, and keep files where R can find them.

1. Install R, then RStudio

R is the language. RStudio (Posit) is an editor that makes R easier to use.

Install them in this order:

1. [R](#) for Windows, macOS, or Linux.
2. [RStudio Desktop](#).

RStudio will not work properly if R is not installed first.

We will work in RStudio, not in the older R application that installs with R.

R was designed for statistics. It is free, it is built around packages, and it is a good language for analysis you can rerun. Python is also widely used in data science. This course uses R.

2. The RStudio Layout

When you open RStudio, you will typically see several panes:

- the **console**, where R evaluates code immediately
- a **source** editor, where you write scripts and documents you want to keep
- an **environment / history** pane, which lists objects in the current session
- a **files / plots / packages / help** pane

The source pane stays closed until you open or create a file.

A few habits that save time:

- The **Files** tab is a limited file manager. You can open, copy, and delete files, and you can set the console working directory from **More**.
 - The **Environment** tab shows objects you have created in this session.
 - The **Help** tab shows documentation. You can also type `help(mean)` or `?mean` in the console.
-

3. Configure RStudio Once

Open **Tools** → **Global Options...** → **General**.

Under **Workspace**:

- uncheck **Restore .RData into workspace at startup**
- set **Save workspace to .RData on exit** to **Never**

If RStudio restores yesterday's objects, you can accidentally use results you no longer have code for. That looks convenient and is bad for reproducibility.

Other useful options:

- **Code** → **Editing**: turn on soft wrapping
 - **Appearance**: choose a font size you can read
-

4. The Console

The console is the pane with the `>` prompt.

Type:

```
1 + 1
```

and press Enter.

R prints:

```
# [1] 2
```

The `[1]` is not part of the answer. It is the index of the first element R is showing you.

The up and down arrow keys cycle through recent console commands. That is useful when you mistype a line.

R is a calculator:

```
3 * 7
9 / 3
(3 + 5) * 6
3 ^ 2
```

`%%` is remainder after integer division:

```
1 %% 2
# 1

4 %% 2
# 0
```

The console is useful for short experiments. It is a poor place to keep your work. When you close RStudio, console history is not a script.

If a command is incomplete, R shows a `+` prompt and waits. Press **Esc** to get back to `>` .

5. Scripts

A **script** is a text file of R code, usually with a `.R` extension.

Create one with **File** → **New File** → **R Script**.

Type this into the script:

```
x <- 10
x * 2
```

Place the cursor on a line and run it with **Ctrl+Enter** (Windows/Linux) or **Cmd+Enter** (macOS). You can also click **Run**.

The code is sent to the console, and the console shows the result.

A useful habit is:

| Type in a script. Run from the script. Treat the console as a scratch pad.

Save the script. You can reopen it in a later session.

Lab 1 is a script. R Markdown and Quarto are for later assignments; see note **1.4**.

6. Comments

R ignores anything after `#` on a line:

```
# This line is only for humans.

x <- 10  # store 10 in x
```

Comments are how you explain *why* you wrote something, not how you hide code you might need later.

7. Assignment

We store a value with `<-` :

```
greeting <- "Hello"
greeting
```

In this course:

- `<-` stores a value in an object
- `=` names an argument inside a function call, for example `mean(x, na.rm = TRUE)`

R is **case sensitive**. `x` and `X` are different objects. `TRUE` is not `true`. `sum()` is not `Sum()`.

8. Getting Help

Almost every function has a help page:

```
help(mean)
?mean
help("%")
```

`??topic` searches help pages. Adding `"in R"` to a web search is often faster than memorizing function names.

9. Organize Your Files

Do not keep every course file on the Desktop with names like `file 1`. You will not be able to tell R where the data is, and other people will not be able to rerun your work.

Create one folder for the course, for example `data_612`, and keep lecture work and assignments in separate subfolders. Use **snake_case** names. Avoid spaces and special characters.

```
data_612/
  class/
    week_01/
  assignments/
    lab_01/
```

Inside a project, a common pattern is:

- `R/` for `.R` scripts
- `data/` for files you import
- `output/` for files you write

You do not need every folder every week. You do need a place that is not a pile.

Back up that folder to Google Drive or another cloud service. A crashed laptop is a common way to lose a course.

10. Where Are Your Files?

R reads and writes files relative to the **working directory**.

```
getwd()
```

In RStudio you can also use **Session** → **Set Working Directory**, or the **Files** pane **More** menu.

For a course project, a better habit is **File** → **New Project**. An RStudio project keeps the working directory attached to that folder, so `read.csv("students.csv")` works when the file is in the project folder. Do not nest one project inside another.

There are two kinds of paths:

- An **absolute path** starts from the drive or user folder, for example `C:/Users/you/Documents/data_612/students.csv`. It works only on your computer. Do not put absolute paths in work you submit.
- A **relative path** starts from the working directory.

```
"students.csv"      # in the working directory
"data/students.csv" # one folder down
"./students.csv"    # same as the first
"..students.csv"    # one folder up
```

Use `/` in paths in R, even on Windows.

If `read.csv("students.csv")` fails, the usual reason is that the file is not in the working directory.

Note **1.2** shows how to import `students.csv`. Lab 1 asks you to import it.

11. Errors Are Information

If R cannot run a line, it prints an error. Read it from the top.

Common first-week mistakes include:

- a missing quotation mark
- a missing parenthesis
- running `install.packages(ggplot2)` without quotes
- calling a function from a package you have not loaded
- looking for a file that is not in the working directory

You do not need to memorize every error. You do need to read it.

12. What to Do Next

Once R and RStudio are installed and you can run a line from a script, continue with:

- **1.2** R Packages and the Tidyverse, including how to import a CSV
- **1.3** Introduction to R Concepts
- **1.4** R Markdown and Quarto
- **Lab 1**, which asks you to predict what R will do *before* you run the code. Work in a `.R` script.